

2. Sortarea prin inserție

$L = [H|T]$

insert $\rightarrow$ sort(T, TS) $\rightarrow$ Lsortat

Program Prolog

domains

lista = real*

predicates

insertsort(lista, lista)

insert(real, lista, lista)

clauses

insertsort([], []).

insertsort([H|T], LSort) :- insertsort(T, TS), insert(H, TS, LSort).

insert(H, [Y|T], [Y|TS]) :- H > Y, insert(H, T, TS), !.

insert(H, L, [H|L]).

Exercițiu: insertsort([2, -1, 7, 11, 8], Lsortat).

Lsortat = [-1, 2, 2, 11, 8]

Program
sort
prolog

insertsort([], []).

insertsort([H|T], LS) :- insertsort(T, TS), insert(H, TS, LS).

insert(H, [Y|T], [Y|TS]) :- H > Y, insert(H, T, TS), !.

insert(H, L, [H|L]).

sort
prolog

3. Metoda Quicksort

- împărțirea listei în funcție de primul element în două liste: o listă cu valori mai mici, respectiv, o listă cu valori mai mari decât primul element
- se sortează cele două liste obținute
- concatenarea listelor sortate

Program Prolog

domains

lista = real *

predicates

qsort(lista, lista)

split(real, lista, lista, lista)

concatenare(lista, lista, lista)

clauses

qsort([], []).

qsort([H|T], LS) :- split(H, T, Lmici, Lmari), qsort(Lmici, LS1), qsort(Lmari, LS2),
concatenare(LS1, [H|LS2], LS).

split(K, [], [], []).

split(K, [H|T], [H|T1], L2) :- H < K, split(K, T, T1, L2), !.

split(K, [H|T], [], [H|T2]) :- split(K, T, [], T2).

concatenare([], L2, L2).

concatenare([H|T], L2, [H|TR]) :- concatenare(T, L2, TR).

execuție: qsort([11, -2, 3, 8, 12], Lsortata).

Lsortata = [-2, 3, 8, 11, 12]